# Analyse et la visualisation des données du Titanic.

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

## Partie 1 : Chargement et exploration

1. Lire le fichier `titanic.csv` avec `pd.read_csv(...)`
2. Afficher :
   - les 10 premières lignes,
   - le nombre de lignes et de colonnes,
   - les noms et types des colonnes,
   - les statistiques descriptives.

### 1. Charger un fichier CSV

In [2]:
```
data = pd.read_csv('Dataset/titanic.csv')
```

### 2. Affichage

### Afficher : Les 10 premières lignes

In [3]:
```
data.head(10)
```

Out[3]:

| | PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 1 | 0 | 3 | Braund, Mr. Owen Harris | male | 22.0 | 1 | 0 | A/5 21171 | 7.2500 | NaN |
| **1** | 2 | 1 | 1 | Cumings, Mrs. John Bradley (Florence Briggs Th... | female | 38.0 | 1 | 0 | PC 17599 | 71.2833 | C85 |
| **2** | 3 | 1 | 3 | Heikkinen, Miss. Laina | female | 26.0 | 0 | 0 | STON/O2. 3101282 | 7.9250 | NaN |
| **3** | 4 | 1 | 1 | Futrelle, Mrs. Jacques Heath (Lily May Peel) | female | 35.0 | 1 | 0 | 113803 | 53.1000 | C123 |
| **4** | 5 | 0 | 3 | Allen, Mr. William Henry | male | 35.0 | 0 | 0 | 373450 | 8.0500 | NaN |
| **5** | 6 | 0 | 3 | Moran, Mr. James | male | NaN | 0 | 0 | 330877 | 8.4583 | NaN |
| **6** | 7 | 0 | 1 | McCarthy, Mr. Timothy J | male | 54.0 | 0 | 0 | 17463 | 51.8625 | E46 |
| **7** | 8 | 0 | 3 | Palsson, Master. Gosta Leonard | male | 2.0 | 3 | 1 | 349909 | 21.0750 | NaN |
| **8** | 9 | 1 | 3 | Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg) | female | 27.0 | 0 | 2 | 347742 | 11.1333 | NaN |
| **9** | 10 | 1 | 2 | Nasser, Mrs. Nicholas (Adele Achem) | female | 14.0 | 1 | 0 | 237736 | 30.0708 | NaN |

## Afficher : le nombre de lignes et de colonnes

In [4]:
```
data.shape
```

Out[4]:
```
(891, 12)
```

## Afficher : Les noms des colonnes

In [5]:
```
data.columns
```

Out[5]:
```
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
       'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],
      dtype='object')
```

## Afficher : Les informations des colonnes

In [6]:
```
data.info
```

Out[6]:
```
<bound method DataFrame.info of      PassengerId  Survived  Pclass  \
0              1         0       3
1              2         1       1
2              3         1       3
3              4         1       1
4              5         0       3
..           ...       ...     ...
886          887         0       2
887          888         1       1
888          889         0       3
889          890         1       1
890          891         0       3

                                                Name     Sex   Age  SibSp  \
0                             Braund, Mr. Owen Harris    male  22.0      1
1    Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                              Heikkinen, Miss. Laina  female  26.0      0
3         Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                            Allen, Mr. William Henry    male  35.0      0
..                                                ...     ...   ...    ...
886                            Montvila, Rev. Juozas    male  27.0      0
887                     Graham, Miss. Margaret Edith  female  19.0      0
888           Johnston, Miss. Catherine Helen "Carrie"  female   NaN      1
889                            Behr, Mr. Karl Howell    male  26.0      0
890                              Dooley, Mr. Patrick    male  32.0      0

     Parch            Ticket     Fare Cabin Embarked
0        0         A/5 21171   7.2500   NaN        S
1        0          PC 17599  71.2833   C85        C
2        0  STON/O2. 3101282   7.9250   NaN        S
3        0            113803  53.1000  C123        S
4        0            373450   8.0500   NaN        S
..     ...               ...      ...   ...      ...
886      0            211536  13.0000   NaN        S
887      0            112053  30.0000   B42        S
888      2        W./C. 6607  23.4500   NaN        S
889      0            111369  30.0000  C148        C
890      0            370376   7.7500   NaN        Q

[891 rows x 12 columns]>
```

## Afficher : les statistiques descriptives

In [7]:
```
data.describe()
```

Out[7]:

| | PassengerId | Survived | Pclass | Age | SibSp | Parch | Fare |
|---|---|---|---|---|---|---|---|
| **count** | 891.000000 | 891.000000 | 891.000000 | 714.000000 | 891.000000 | 891.000000 | 891.000000 |
| **mean** | 446.000000 | 0.383838 | 2.308642 | 29.699118 | 0.523008 | 0.381594 | 32.204208 |
| **std** | 257.353842 | 0.486592 | 0.836071 | 14.526497 | 1.102743 | 0.806057 | 49.693429 |
| **min** | 1.000000 | 0.000000 | 1.000000 | 0.420000 | 0.000000 | 0.000000 | 0.000000 |
| **25%** | 223.500000 | 0.000000 | 2.000000 | 20.125000 | 0.000000 | 0.000000 | 7.910400 |
| **50%** | 446.000000 | 0.000000 | 3.000000 | 28.000000 | 0.000000 | 0.000000 | 14.454200 |
| **75%** | 668.500000 | 1.000000 | 3.000000 | 38.000000 | 1.000000 | 0.000000 | 31.000000 |
| **max** | 891.000000 | 1.000000 | 3.000000 | 80.000000 | 8.000000 | 6.000000 | 512.329200 |

# Partie 2 : Nettoyage et préparation

1. Identifier les colonnes avec des valeurs manquantes.
2. Remplacer les âges manquants par la moyenne des âges.
3. Supprimer les lignes où **Embarked** est manquant.
4. Créer une nouvelle colonne **FamilySize** = SibSp + Parch + 1.

## 1. Identifier les colonnes avec des valeurs nulles

In [8]:
```
print(data.isnull().sum())
```

```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

## 2. Remplacer les âges manquants par la moyenne des âges.

In [9]:
```
data["Age"] = data["Age"].fillna(data["Age"].mean())
```

In [10]:
```
print(data.isnull().sum())
```

```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age              0
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

## 3. Supprimer les lignes où Embarked est manquant.

In [11]:
```
data.shape
```

Out[11]:
```
(891, 12)
```

In [12]:
```
# Supprimer lignes sans 'Embarked'
data.dropna(subset=["Embarked"], inplace=True)
```

In [13]:
```
print(data.isnull().sum())
```

```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age              0
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         0
dtype: int64
```

In [14]:
```
data
```

Out[14]:

| | PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare |
|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 1 | 0 | 3 | Braund, Mr. Owen Harris | male | 22.000000 | 1 | 0 | A/5 21171 | 7.2500 |
| **1** | 2 | 1 | 1 | Cumings, Mrs. John Bradley (Florence Briggs Th... | female | 38.000000 | 1 | 0 | PC 17599 | 71.2833 |
| **2** | 3 | 1 | 3 | Heikkinen, Miss. Laina | female | 26.000000 | 0 | 0 | STON/O2. 3101282 | 7.9250 |
| **3** | 4 | 1 | 1 | Futrelle, Mrs. Jacques Heath (Lily May Peel) | female | 35.000000 | 1 | 0 | 113803 | 53.1000 |
| **4** | 5 | 0 | 3 | Allen, Mr. William Henry | male | 35.000000 | 0 | 0 | 373450 | 8.0500 |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| **886** | 887 | 0 | 2 | Montvila, Rev. Juozas | male | 27.000000 | 0 | 0 | 211536 | 13.0000 |
| **887** | 888 | 1 | 1 | Graham, Miss. Margaret Edith | female | 19.000000 | 0 | 0 | 112053 | 30.0000 |
| **888** | 889 | 0 | 3 | Johnston, Miss. Catherine Helen "Carrie" | female | 29.699118 | 1 | 2 | W./C. 6607 | 23.4500 |
| **889** | 890 | 1 | 1 | Behr, Mr. Karl Howell | male | 26.000000 | 0 | 0 | 111369 | 30.0000 |
| **890** | 891 | 0 | 3 | Dooley, Mr. Patrick | male | 32.000000 | 0 | 0 | 370376 | 7.7500 |

889 rows × 12 columns

## 4. Créer une nouvelle colonne FamilySize = SibSp + Parch + 1.

**Explication**

**SibSp** : nombre de frères/sœurs + conjoints à bord.

**Parch** : nombre de parents + enfants à bord.

$+1$ : on ajoute le passager lui-même.

Ainsi, **FamilySize** représente la taille totale de la famille à bord (y compris la personne).

In [15]:
```
data.loc[:, "FamilySize"] = data["SibSp"] + data["Parch"] + 1

## data["FamilySize"] = data["SibSp"] + data["Parch"] + 1
```

In [16]:
```
data
```

Out[16]:

| | PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare |
|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 1 | 0 | 3 | Braund, Mr. Owen Harris | male | 22.000000 | 1 | 0 | A/5 21171 | 7.2500 |
| **1** | 2 | 1 | 1 | Cumings, Mrs. John Bradley (Florence Briggs Th... | female | 38.000000 | 1 | 0 | PC 17599 | 71.2833 |
| **2** | 3 | 1 | 3 | Heikkinen, Miss. Laina | female | 26.000000 | 0 | 0 | STON/O2. 3101282 | 7.9250 |
| **3** | 4 | 1 | 1 | Futrelle, Mrs. Jacques Heath (Lily May Peel) | female | 35.000000 | 1 | 0 | 113803 | 53.1000 |
| **4** | 5 | 0 | 3 | Allen, Mr. William Henry | male | 35.000000 | 0 | 0 | 373450 | 8.0500 |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| **886** | 887 | 0 | 2 | Montvila, Rev. Juozas | male | 27.000000 | 0 | 0 | 211536 | 13.0000 |
| **887** | 888 | 1 | 1 | Graham, Miss. Margaret Edith | female | 19.000000 | 0 | 0 | 112053 | 30.0000 |
| **888** | 889 | 0 | 3 | Johnston, Miss. Catherine Helen "Carrie" | female | 29.699118 | 1 | 2 | W./C. 6607 | 23.4500 |
| **889** | 890 | 1 | 1 | Behr, Mr. Karl Howell | male | 26.000000 | 0 | 0 | 111369 | 30.0000 |
| **890** | 891 | 0 | 3 | Dooley, Mr. Patrick | male | 32.000000 | 0 | 0 | 370376 | 7.7500 |

889 rows × 13 columns

# Partie 3 : Analyse statistique

1. Calculer l'âge moyen par classe (**Pclass**),
2. Calculer l'âge moyen par (**Sex**),
3. Calculer le tarif moyen (**Fare**) par sexe (**Sex**),
4. Calculer le tarif moyen (**Fare**) par classe (**Pclass**) et port d'embarquement (**Embarked**),

5. Calculer le taux de survie (**Survived**) par port d'embarquement (**Embarked**),
6. Calculer le taux de survie (**Survived**) par classe (**Pclass**),
7. Trouver le passager le plus âgé (nom et âge).

## 1. Calculer l'âge moyen par classe (Pclass)

In [17]:
```
data.groupby("Pclass")["Age"].mean()
```

Out[17]:
```
Pclass
1    36.927073
2    29.866958
3    26.403259
Name: Age, dtype: float64
```

## 2. Calculer l'âge moyen par (Sex)

In [18]:
```
data.groupby("Sex")["Age"].mean()
```

Out[18]:
```
Sex
female    28.077094
male      30.505824
Name: Age, dtype: float64
```

## 3. Calculer le tarif moyen (Fare) par sexe (Sex)

In [19]:
```
print(data.groupby(["Sex"])["Fare"].mean())
```

```
Sex
female    44.252124
male      25.523893
Name: Fare, dtype: float64
```

## 4. Calculer le tarif moyen (Fare) par classe (Pclass) et port d'embarquement (Embarked)

In [20]:
```
print(data.groupby(["Pclass", "Embarked"])["Fare"].mean())
```

```
Pclass  Embarked
1       C           104.718529
        Q            90.000000
        S            70.364862
2       C            25.358335
        Q            12.350000
        S            20.327439
3       C            11.214083
        Q            11.183393
        S            14.644083
Name: Fare, dtype: float64
```

## 5. Calculer le taux de survie (Survived) par port d'embarquement (Embarked)

In [21]:
```
print(data.groupby("Embarked")["Survived"].mean() * 100 )
```

```
Embarked
C    55.357143
Q    38.961039
S    33.695652
Name: Survived, dtype: float64
```

### 6. Calculer le taux de survie (Survived) par classe (Pclass)

In [22]:
```
print(data.groupby("Pclass")["Survived"].mean() * 100 )
```

```
Pclass
1    62.616822
2    47.282609
3    24.236253
Name: Survived, dtype: float64
```

### 7. Trouver le passager le plus âgé (nom et âge).

In [23]:
```
data["Age"].idxmax()
```

Out[23]:
```
630
```

In [24]:
```
print(data.loc[data["Age"].idxmax(), ["Name", "Age"]])
```

```
Name    Barkworth, Mr. Algernon Henry Wilson
Age                                     80.0
Name: 630, dtype: object
```

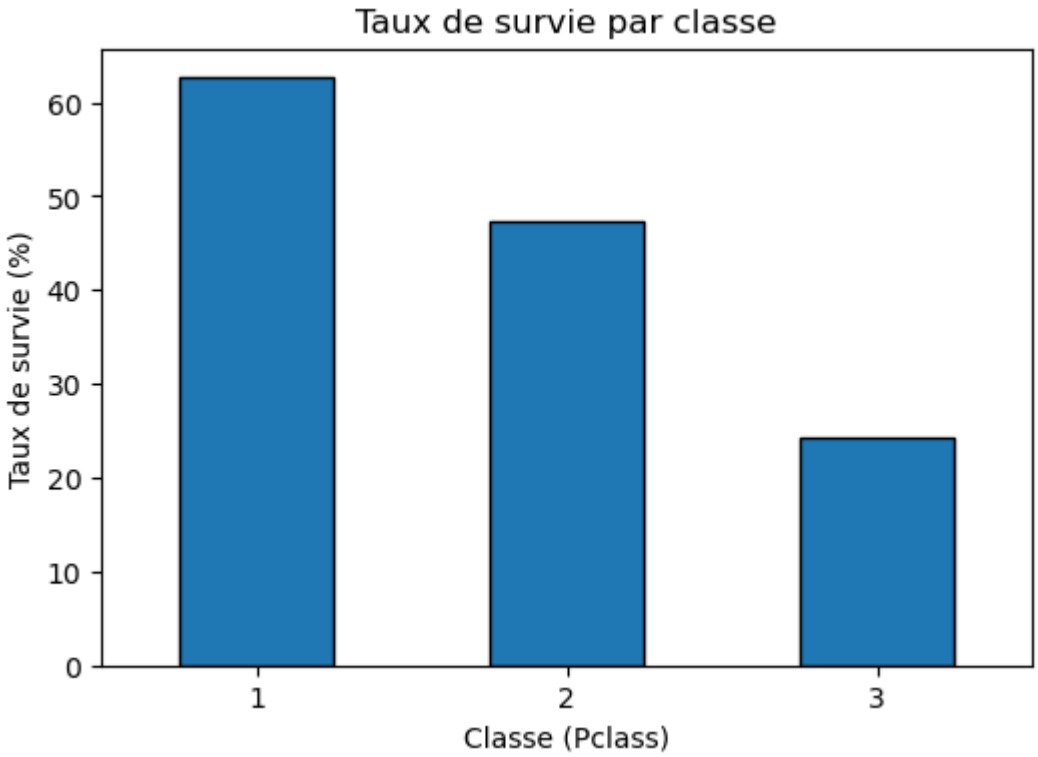
## Partie 4 : Visualisation

1. Tracer un diagramme à barres du taux de survie par classe.
2. Tracer l'histogramme de la distribution des âges.
3. Tracer une heatmap (ou tableau) de corrélations entre **Age**, **Fare**, **Pclass**, **Survived**.

### 1. Tracer un diagramme à barres du taux de survie par classe.

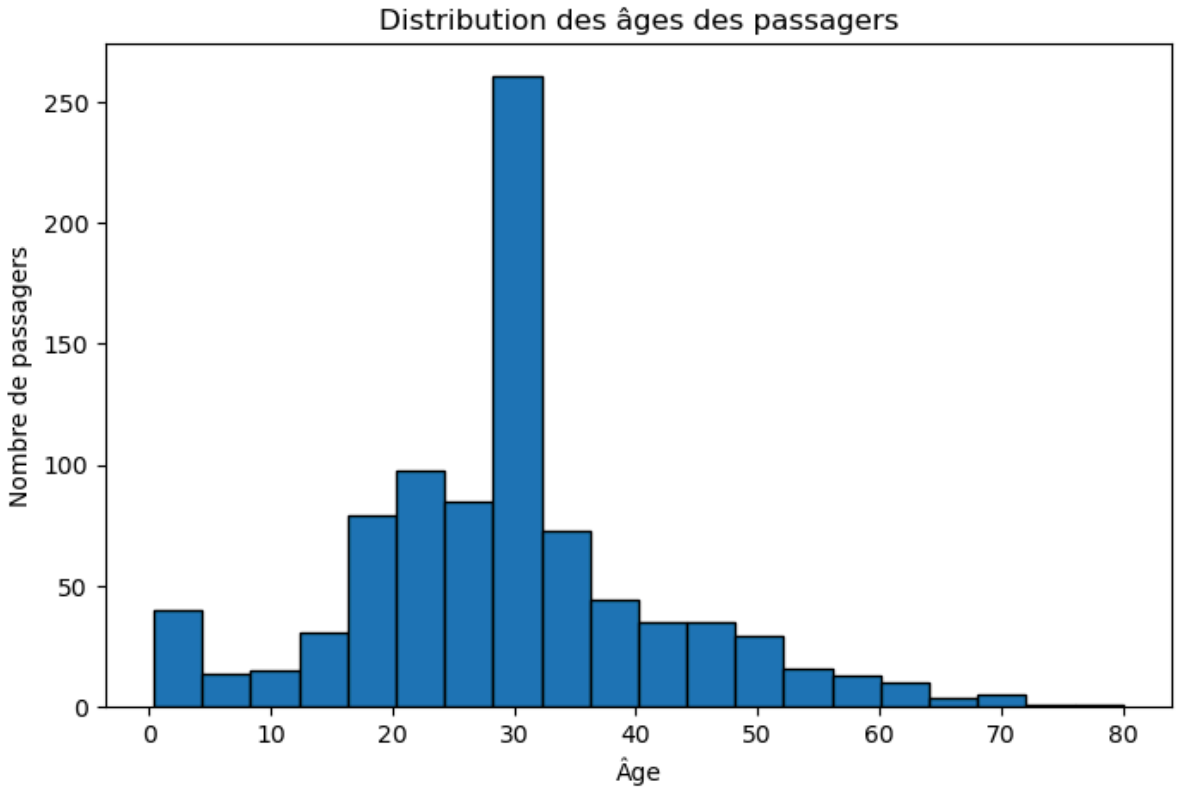
In [25]:
```
# Calcul du taux de survie par classe
survival_rate_by_class = data.groupby("Pclass")["Survived"].mean() * 100

# Tracé du diagramme à barres
plt.figure(figsize=(6,4))
survival_rate_by_class.plot(kind="bar", edgecolor="black")
plt.title("Taux de survie par classe")
plt.xlabel("Classe (Pclass)")
plt.ylabel("Taux de survie (%)")
plt.xticks(rotation=0)
plt.show()
```




## 2. Tracer l'histogramme de la distribution des âges.

In [26]:
```
plt.figure(figsize=(8,5))
plt.hist(data["Age"], bins=20, edgecolor="black")
plt.title("Distribution des âges des passagers")
plt.xlabel("Âge")
plt.ylabel("Nombre de passagers")
plt.show()
```



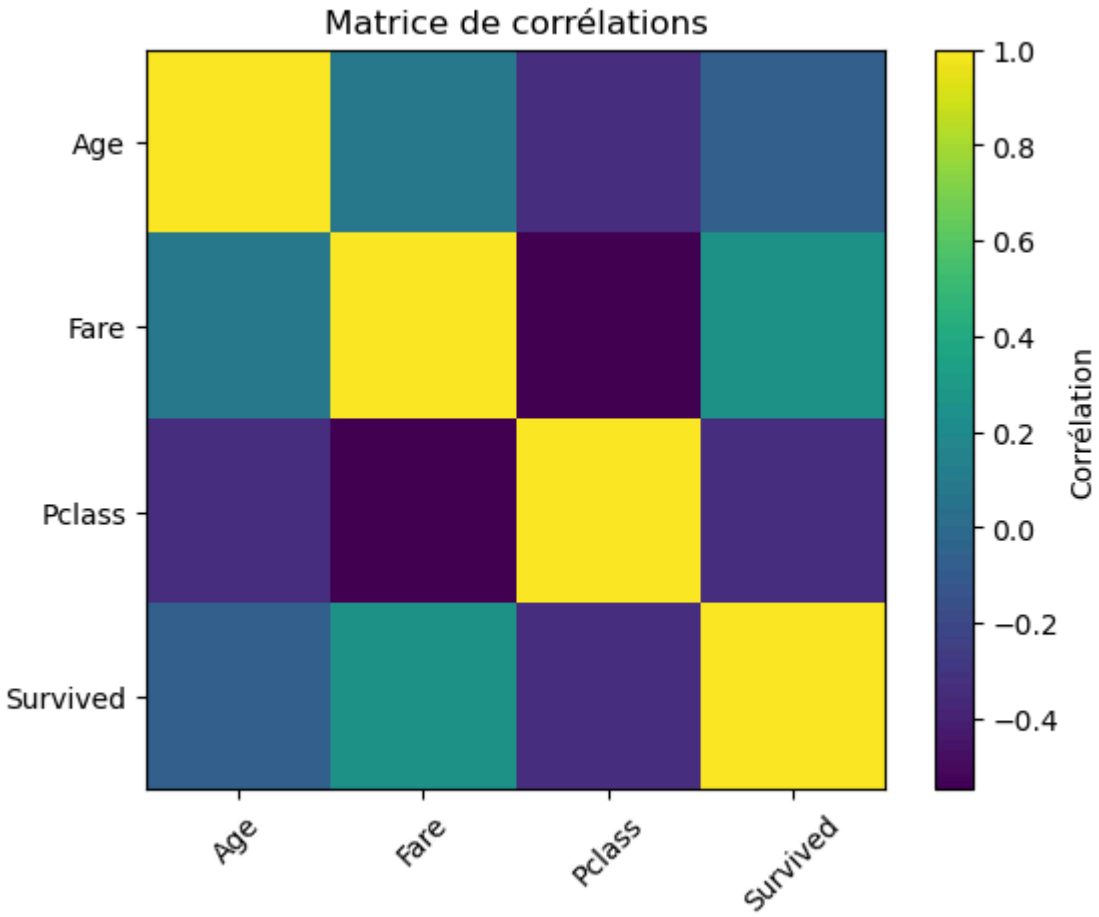

## 3. Tracer une heatmap (ou tableau) de corrélations entre Age, Fare, Pclass, Survived.

In [27]:
```
# Sélection des colonnes d'intérêt
cols = ["Age", "Fare", "Pclass", "Survived"]
corr_matrix = data[cols].corr()

# Affichage en tableau
print(corr_matrix)

# Heatmap simple avec matplotlib
plt.imshow(corr_matrix)
plt.colorbar(label="Corrélation")
plt.xticks(range(len(cols)), cols, rotation=45)
plt.yticks(range(len(cols)), cols)
plt.title("Matrice de corrélations")
plt.show()
```

```
               Age      Fare    Pclass  Survived
Age       1.000000  0.088604 -0.327954 -0.074673
Fare      0.088604  1.000000 -0.548193  0.255290
Pclass   -0.327954 -0.548193  1.000000 -0.335549
Survived -0.074673  0.255290 -0.335549  1.000000
```



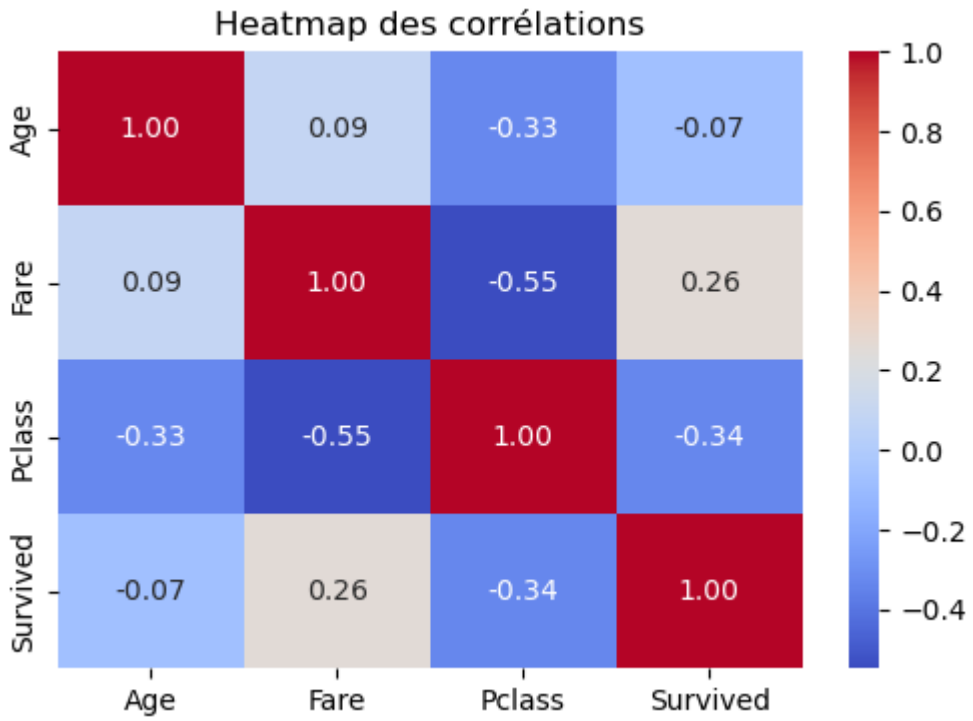

## Version avec seaborn

In [28]:
```
import seaborn as sns

# Calcul de la matrice de corrélations
cols = ["Age", "Fare", "Pclass", "Survived"]
corr_matrix = data[cols].corr()

# Heatmap
plt.figure(figsize=(6,4))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f")
```

```
plt.title("Heatmap des corrélations")
plt.show()
```




In [ ]: